MANCHESTER 1824
The University of Manchester

# Using Snakemake to develop workflows for bioinformatics analysis

## 10 November 2026

**Raymond Wan**
**Manchester Institute of Biotechnology &**
**Department of Physics and Astronomy**

MANCHESTER 1824
The University of Manchester

# Outline

Introduction

Background

Basic structure

Final example

Tips

Summary

MANCHESTER 1824
The University of Manchester

# Overview

Snakemake is:

- a workflow engine based on Python
- useful for applying a series of commands (i.e., a “pipeline”)

MANCHESTER 1824
The University of Manchester

# Overview

Snakemake is:

- a workflow engine based on Python
- useful for applying a series of commands (i.e., a “pipeline”)



(Figure source: Wikipedia: pipeline.)

MANCHESTER 1824
The University of Manchester

# Overview

BIOINFORMATICS APPLICATION NOTE

Vol. 28 no. 19 2012, pages 2520–2522
doi:10.1093/bioinformatics/bts480

*Genome analysis*

Advance Access publication August 20, 2012

## Snakemake—a scalable bioinformatics workflow engine

Johannes Köster[1,2,*] and Sven Rahmann[1]

[1]Genome Informatics, Institute of Human Genetics, University of Duisburg-Essen and [2]Paediatric Oncology, University Childrens Hospital, 45147 Essen, Germany

Associate Editor: Alfonso Valencia

MANCHESTER 1824
The University of Manchester

# Overview

BIOINFORMATICS **APPLICATION NOTE**

*Vol. 28 no. 19 2012, pages 2520–2522*
*doi:10.1093/bioinformatics/bts480*

*Genome analysis*

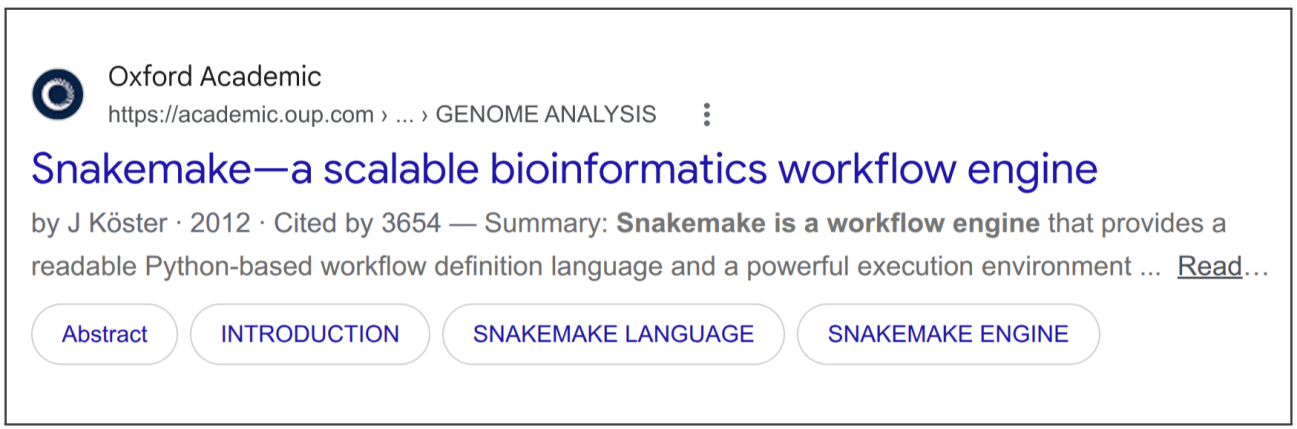
Advance Access publication August 20, 2012

## Snakemake—a scalable bioinformatics workflow engine

Johannes Köster[1,2,*] and Sven Rahmann[1]

[1]Genome Informatics, Institute of Human Genetics, University of Duisburg-Essen and [2]Paediatric Oncology, University Childrens Hospital, 45147 Essen, Germany

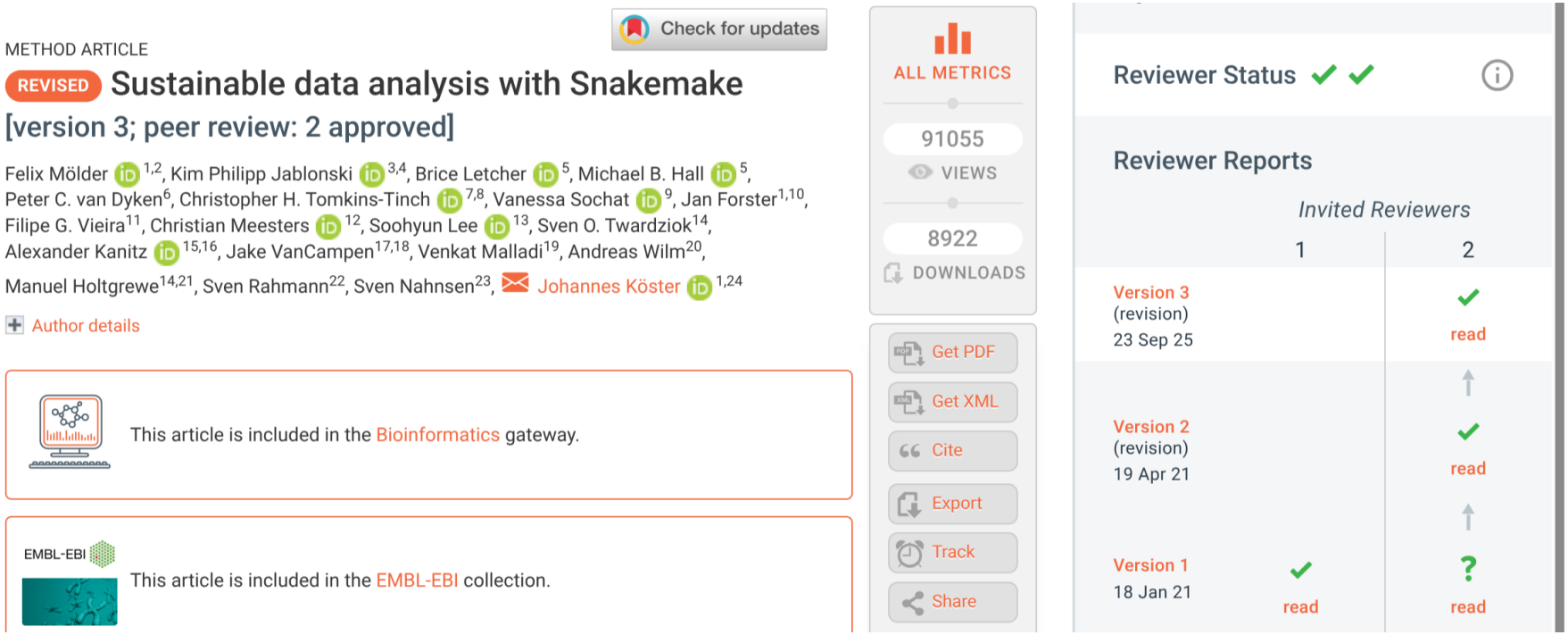
Associate Editor: Alfonso Valencia




(Figure source: (Köster and Rahmann, 2012).)

# Overview

Currently, Snakemake developers encouraging citing this “rolling” publication:



METHOD ARTICLE

REVISED **Sustainable data analysis with Snakemake**

[version 3; peer review: 2 approved]

Felix Mölder[1,2], Kim Philipp Jablonski[3,4], Brice Letcher[5], Michael B. Hall[5], Peter C. van Dyken[6], Christopher H. Tomkins-Tinch[7,8], Vanessa Sochat[9], Jan Forster[1,10], Filipe G. Vieira[11], Christian Meesters[12], Soohyun Lee[13], Sven O. Twardziok[14], Alexander Kanitz[15,16], Jake VanCampen[17,18], Venkat Malladi[19], Andreas Wilm[20], Manuel Holtgrewe[14,21], Sven Rahmann[22], Sven Nahnsen[23], Johannes Köster[1,24]

Check for updates

Author details

This article is included in the Bioinformatics gateway.

EMBL-EBI

This article is included in the EMBL-EBI collection.

ALL METRICS

91055 VIEWS

8922 DOWNLOADS

Get PDF

Get XML

Cite

Export

Track

Share

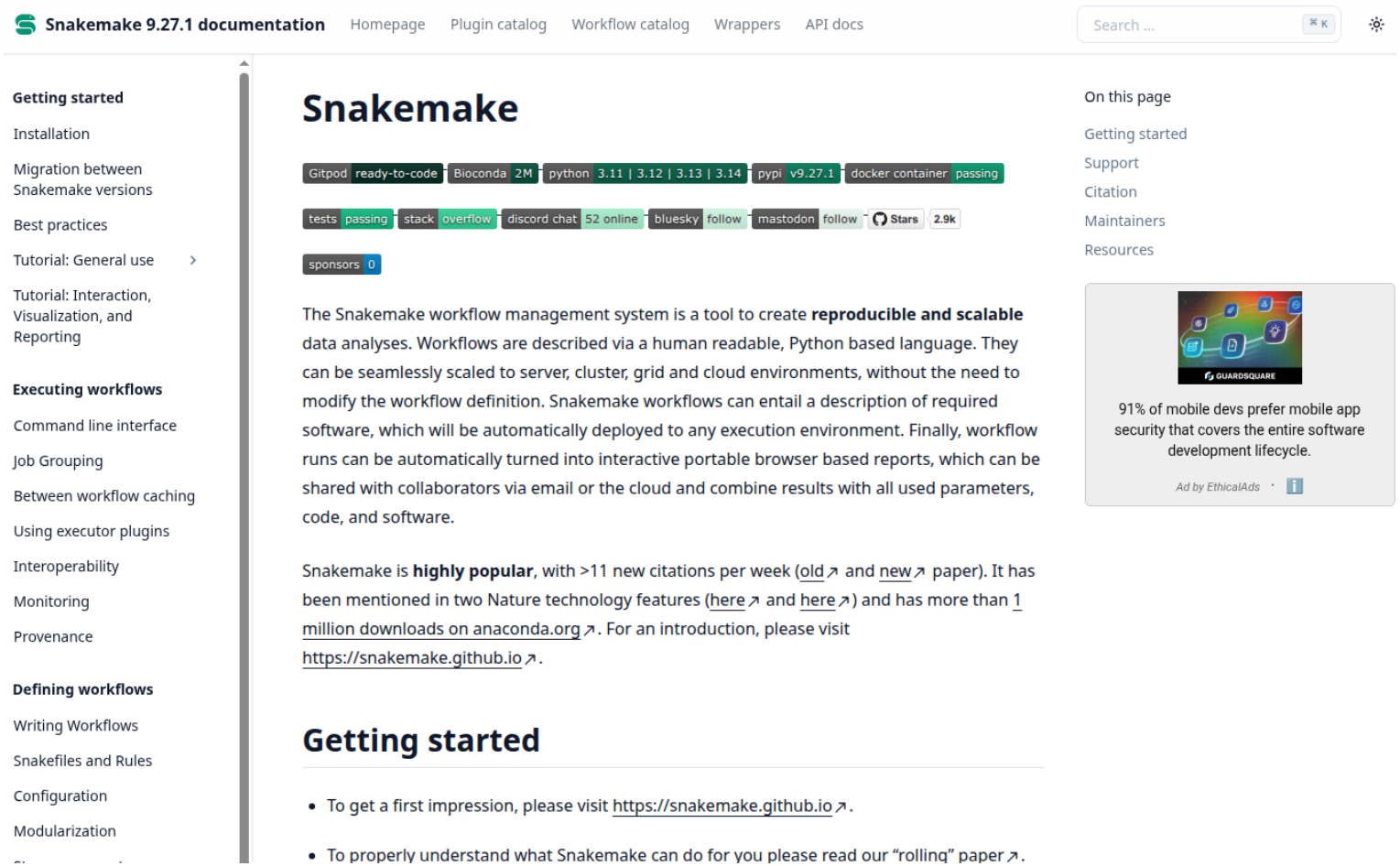
Reviewer Status ✓ ✓

Reviewer Reports

| | Invited Reviewers | |
|---|---|---|
| | 1 | 2 |
| Version 3 (revision) 23 Sep 25 | | ✓ read |
| Version 2 (revision) 19 Apr 21 | | ✓ read |
| Version 1 18 Jan 21 | ✓ read | ? read |

(Figure source: (Mölder et al., 2025).)

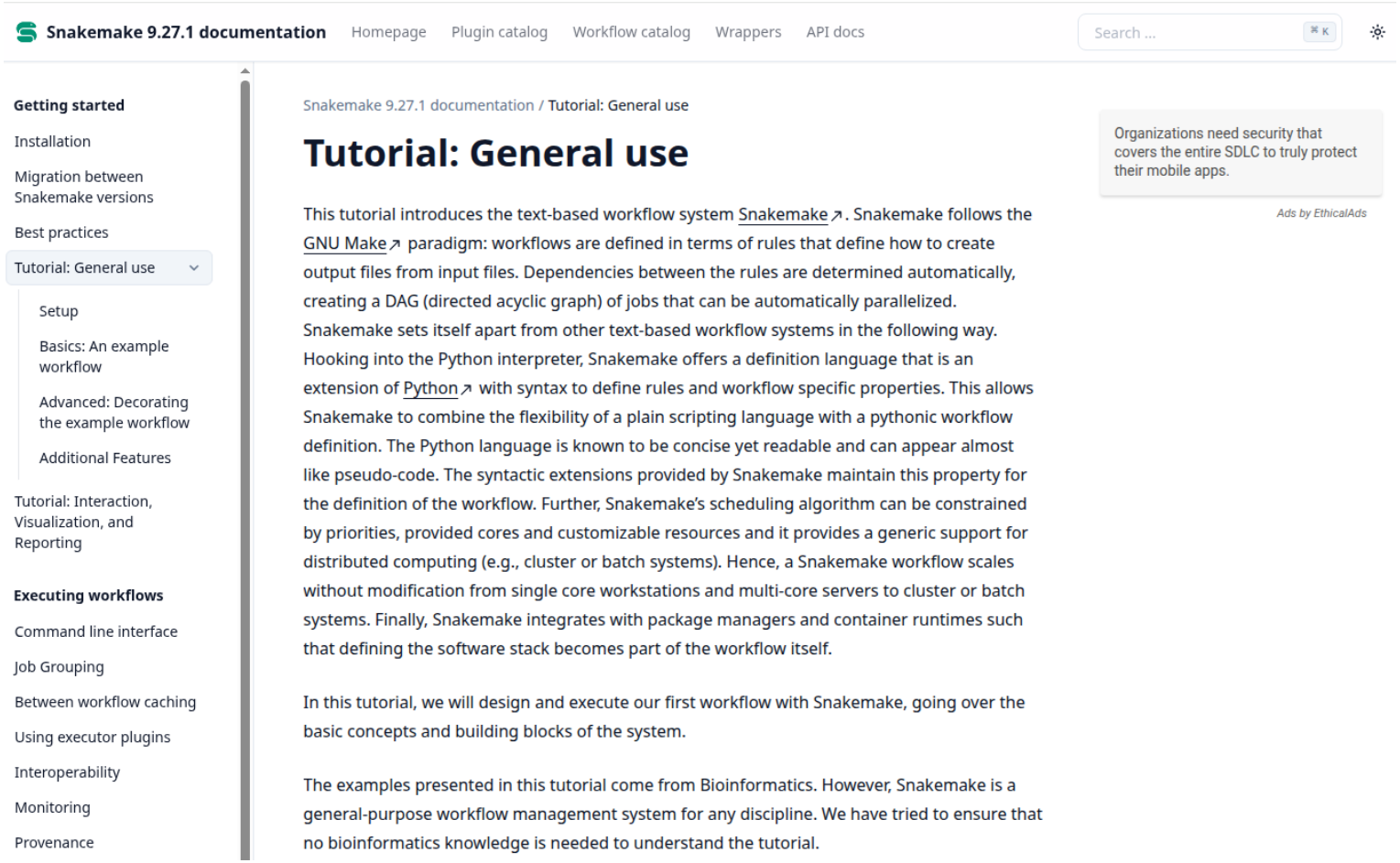
# Snakemake's homepage




(Figure source: https://snakemake.readthedocs.io/en/stable/.)

MANCHESTER 1824
The University of Manchester

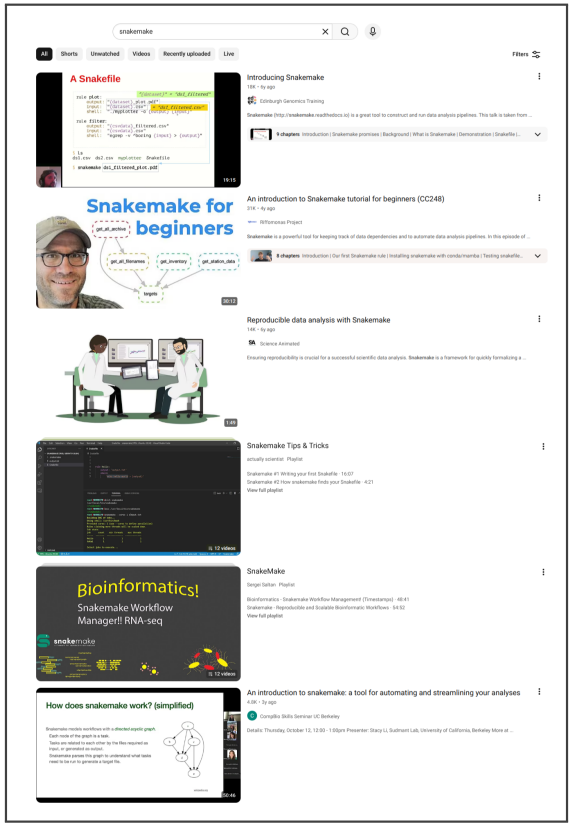
# Snakemake's tutorial




(Figure source: https://snakemake.readthedocs.io/en/stable/tutorial/tutorial.html.)

MANCHESTER 1824
The University of Manchester

# YouTube videos




So, a lot of information on the Internet now...

MANCHESTER 1824
The University of Manchester

# My story

By 2016, I had been working in bioinformatics for >10 years...

- Quite stuck in my ways for workflows (i.e., `bash` scripts).
- A new colleague joined our lab with a background in CS. Asked me where to start. I had recently came across the Snakemake paper and gave it to him since he also knew Python.
- A fast learner, he picked it up in a week. And he kept telling me how great it was.
- Months later, I switched, spending just over a week changing what I had been doing from `bash` to Snakemake.

Of course, one of the best decisions I have made as a bioinformatician...

MANCHESTER 1824
The University of Manchester

# Talk's motivation

Motivation of this talk:

- I assume that you have a workflow / need to develop a workflow.
- There are so many languages and tools out there... ☹
- You are wondering if Snakemake is "worth your time"...

Everything might be too fast today. My aim is to provide a "bird's eye" view so that (hopefully) you have enough information to decide if you should invest more time.

If it is what you are looking for, then you have the Snakemake homepage, YouTube videos, etc. as your next resource.

MANCHESTER 1824
The University of Manchester

# Talk's motivation

"Disclaimer":

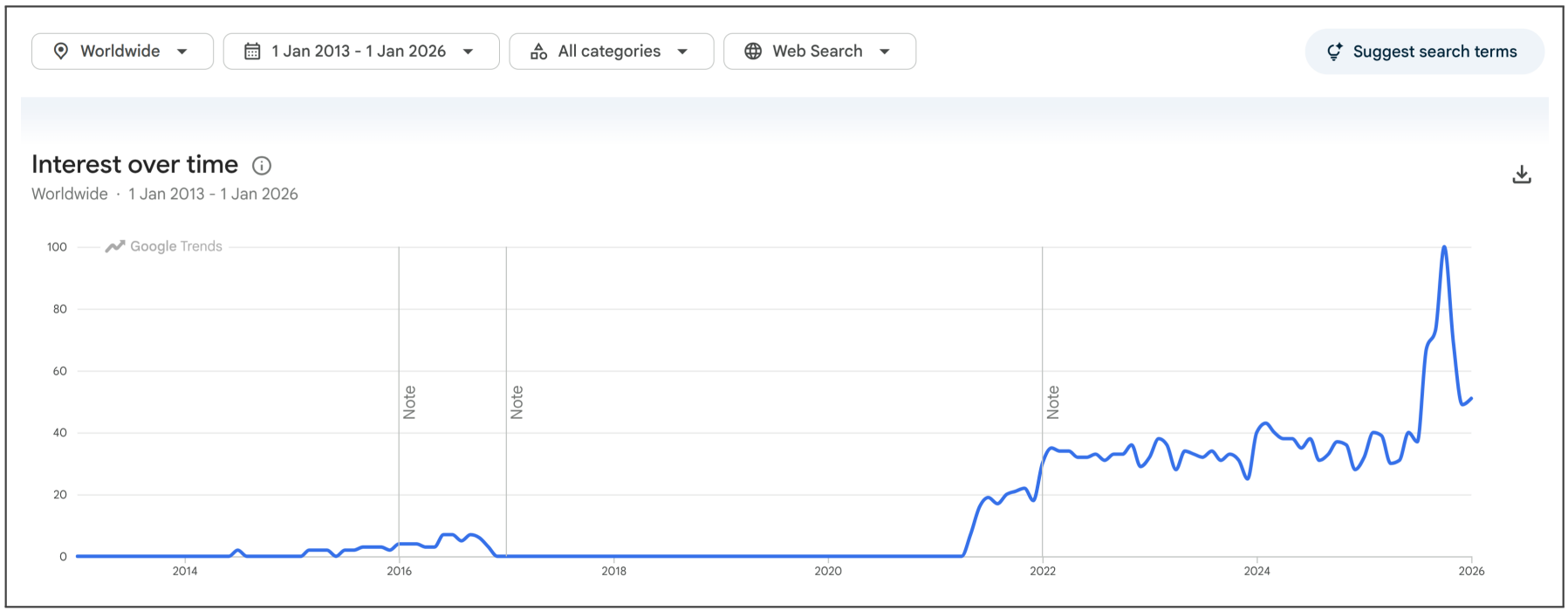
- Despite using it for 10 years, I only know what I needed. I have not yet explored many (newer...) features about Snakemake.
- Have been happy with it since 2016; did not look into alternatives. No doubt there are better alternatives out there!

Alternatives:

- NextFlow
- Common Workflow Language (CWL)
- Galaxy (i.e., web browser based)

MANCHESTER 1824
The University of Manchester

# Google trends



MANCHESTER 1824
The University of Manchester

# Overview

- Background to the problem
- Basic structure of Snakemake files through working examples
- A final “larger” example
- Some random tips
- Summary
  - Answers to questions that people have asked me before

If you find these slides useful (or if I am talking too fast...), my last slide has a QR code to these PDF slides.

MANCHESTER 1824
The University of Manchester

# Outline

Introduction

Background

Basic structure

Final example

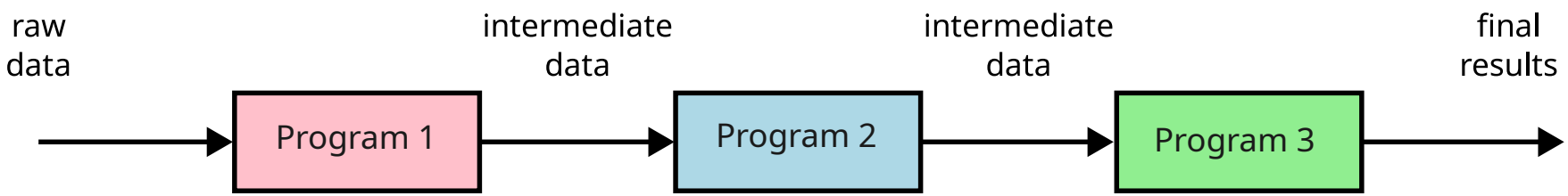
Tips

Summary

MANCHESTER 1824
The University of Manchester

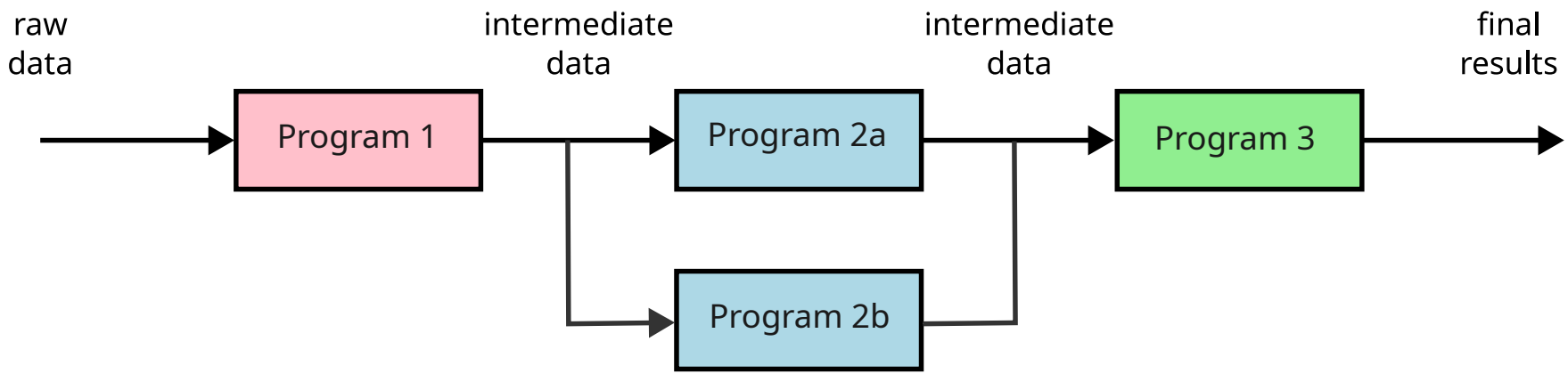
# Pipeline

Sequence of programs, running one after the other:



# Pipeline

Branching is also possible:



MANCHESTER 1824
The University of Manchester

# Counting characters

Running a sequence of programs is not new to bioinformatics.

Suppose we wanted to count the number of words and characters in the first 1000 lines of the file `alice29`.

In Linux, we can do the following with a temporary file:

```
head -n 1000 alice29 >tmp1.txt
wc -w -c tmp1.txt >alice29.count
```

MANCHESTER 1824
The University of Manchester

# Counting characters

Running a sequence of programs is not new to bioinformatics.

Suppose we wanted to count the number of words and characters in the first 1000 lines of the file `alice29`.

In Linux, we can do the following with a temporary file:

```
head -n 1000 alice29 >tmp1.txt
wc -w -c tmp1.txt >alice29.count
```

Or, we can use the vertical bar or “pipe” character (|) as follows:

```
cat alice29 | head -n 1000 | wc -w -c >alice29.count
```

# Make

The program `Make` takes this one step further by allowing users to manage dependencies.

A file called `Makefile` is read by `Make`.

It can indicate that `alice29.count` is updated only if `alice29` was updated.

MANCHESTER 1824
The University of Manchester

# Make

WIKIPEDIA
25 years of the free encyclopedia

Search Wikipedia | Search

## Make (software)

28 languages

Article | Talk | Read | Edit | View history

Contents hide


(Top)
Origin
Variants
Use
Makefile
Rules
Macros
Suffix rules
Pattern rules
Comment
Directive
Line continuation
Examples
Dependency tracking
See also
References
External links


From Wikipedia, the free encyclopedia

In software development, **Make** is a command-line interface software tool that performs actions ordered by configured dependencies as defined in a configuration file called a *makefile*. It is commonly used for build automation to build executable code (such as a program or library) from source code. Make is also not limited to building and can perform any operation available via the operating system shell.

Make is widely used, especially in Unix and Unix-like operating systems, even though many competing technologies and tools are available, including similar tools that perform actions based on dependencies, some compilers and interactively via an integrated development environment.

In addition to referring to the original Unix tool, Make is also a technology since multiple tools have been implemented with roughly the same functionality – including similar makefile syntax and semantics.

| Make | |
|---|---|
| **Paradigm** | macro, declarative |
| **Designed by** | Stuart Feldman |
| **First appeared** | April 1976; 50 years ago |
| **Implementation language** | C |
| **OS** | Unix-like, Inferno |
| **File formats** | Makefile |
| **Major implementations** | |
| BSD, GNU, nmake | |
| **Dialects** | |
| BSD make, GNU make, Microsoft nmake | |
| **Influenced** | |
| Ant, Rake, MSBuild, and *others* | |

### Origin [edit]

Stuart Feldman created Make while at Bell Labs. An early version was completed in April 1976.[1][2][3] Feldman received the 2003 ACM Software System Award for authoring Make.[4]

Feldman describes the inspiration to write Make as arising from a coworker's frustration with the available tooling of the time:

# Makefile

Typically, a `Makefile` has a series of rules. One rule looks like this:

```
target: prerequisites
  command
  ⋮
  command
```

→ If the dates of the files in the prerequisites are newer than the target, then update the target by running the commands (which are preceded by a single tab character).

MANCHESTER 1824
The University of Manchester

# “Make” example

As a series of commands:

```
alice29.count: alice29
  head -n 1000 alice29 >tmp1.txt
  wc -w -c tmp1.txt >alice29.count
```

MANCHESTER 1824
The University of Manchester

# “Make” example

As a series of commands:

```
alice29.count: alice29
  head -n 1000 alice29 >tmp1.txt
  wc -w -c tmp1.txt >alice29.count
```

As one long command with pipes:

```
alice29.count: alice29
  cat alice29 | head -n 1000 | wc -w -c >alice29.count
```

MANCHESTER 1824
The University of Manchester

# “Make” example[1]

```
$ ls
Makefile
$ make
cat ../alice29 | head -n 1000 | wc -w -c >alice29.count
$ ls
alice29.count  Makefile
$ make
make: 'alice29.count' is up to date.
$ cat alice29.count
   8457   47564
$
```

[1] V1

MANCHESTER 1824
The University of Manchester

# “Make” now

Make and Makefiles have not disappeared:

- The “make” program still exists; and to keep something trivial up-to-date, it remains very useful.

Many similar tools have since existed that have “Make” in their names:

- CMake
- qmake
- Snakemake

# Make

Their purposes:

- CMake – Cross-platform make, for projects made with C/C++ (for example)
- qmake – Qt projects make, for C++ projects using Qt to make a graphical user interface
- Snakemake – Development of workflows with Python syntax

MANCHESTER 1824
The University of Manchester

# Snakemake

So, that is where the “make” part of Snakemake comes from!

The “Snake” part is due to Snakemake being made with Python and borrows some of Python’s syntax.

But it is a separate program with a different file format. i.e., Snakemake files are not Python programs.

MANCHESTER 1824
The University of Manchester

# Data sets

Two sets of text files will be considered. Both are used to test text compression systems (i.e., my former area of research):

- Calgary corpus from the University of Calgary (T. C. Bell, Cleary, and Witten, 1990).
- Canterbury corpus from the University of Canterbury (Arnold and T. Bell, 1997).

While not bioinformatics related, we are all familiar with text and both are publicly available.

Even so, I have taken only the first 1,024 lines of each since the originals are too large for our purposes.

MANCHESTER 1824
The University of Manchester

# Outline

Introduction

Background

Basic structure

Final example

Tips

Summary

MANCHESTER 1824
The University of Manchester

# Basic format

Like Make, a Snakemake file consists of a series of rules

- While the format is different from Make, each rule has:
  - A name
  - Series of directives:
    1. Input(s)
    2. Output(s)
    3. Command(s) to run to convert the inputs to the outputs
    4. ...
- Rules can be chained together and/or create branches.
- A rule is applied if the date/time of the input(s) is newer than the output(s).

MANCHESTER 1824
The University of Manchester

# Snakemake program

The program is simply `snakemake`.

If the program is not installed on the machine you are using, then see the documentation on how to install it.

If you administer your own Linux machine, then you can install it with just `sudo apt-get install snakemake`.

Ubuntu 26.04 LTS will install version 7.32.4. The latest version appears to be 9.27.1.

MANCHESTER 1824
The University of Manchester

# Filename

Like `Make`, when you run `snakemake`, it will search for a file of rules. This file should be one of the following:

- `Snakefile`,
- `snakefile`,
- `workflow/Snakefile`,
- `workflow/snakefile`

However, I tend to use the `--snakefile` option to specify my own filename: `Snakefile.smk` (so that I can find my Snakemake files easily).

Like Python scripts which can be broken up into modules and "imported" in, the rules in a Snakemake file can be separated into multiple files and "included" in.

MANCHESTER 1824
The University of Manchester

# Snakemake options

See the manual for a list of options. But besides `--snakefile`, I find these useful:

- `--cores` – number of cores to use; rules that are not dependent on each other can execute in parallel across multiple cores
- `--dryrun` – perform a "dry run" (i.e., do not actually run)
- `-p` – print shell commands

# Sample Rule[2]

```
DATA_DIR = ”../../data”
OUTPUT_DIR = ”Output”

rule CountAlice:
    input: DATA_DIR + ”/canterbury/alice29”
    output: OUTPUT_DIR + ”/alice29.count”
    shell:
        ”””
        cat {input} | head -n 1000 | wc -w -c >{output}
        ”””
```



Note: Variables are enclosed in curly braces.

[2] V2

# Sample Rule[2]

```
DATA_DIR = ”../../data”
OUTPUT_DIR = ”Output”

rule CountAlice:
    input: DATA_DIR + ”/canterbury/alice29”
    output: OUTPUT_DIR + ”/alice29.count”
    shell:
        ”””
        cat {input} | head -n 1000 | wc -w -c >{output}
        ”””
```



Note: Variables are enclosed in curly braces.

[2] V2

# Sample Rule[2]

```
DATA_DIR = ”../../data”
OUTPUT_DIR = ”Output”

rule CountAlice:
    input: DATA_DIR + ”/canterbury/alice29”
    output: OUTPUT_DIR + ”/alice29.count”
    shell:
        ”””
        cat {input} | head -n 1000 | wc -w -c >{output}
        ”””
```



Note: Variables are enclosed in curly braces.

[2] V2

# Sample Rule[2]

```
DATA_DIR = ”../../data”
OUTPUT_DIR = ”Output”

rule CountAlice:
    input: DATA_DIR + ”/canterbury/alice29”
    output: OUTPUT_DIR + ”/alice29.count”
    shell:
        ”””
        cat {input} | head -n 1000 | wc -w -c >{output}
        ”””
```



Note: Variables are enclosed in curly braces.

[2] V2

# Sample Rule[2]

```
DATA_DIR = ”../../data”
OUTPUT_DIR = ”Output”

rule CountAlice:
    input: DATA_DIR + ”/canterbury/alice29”
    output: OUTPUT_DIR + ”/alice29.count”
    shell:
        ”””
        cat {input} | head -n 1000 | wc -w -c >{output}
        ”””
```

Note: Variables are enclosed in curly braces.

[2] V2

MANCHESTER 1824
The University of Manchester

# Sample Rule[2]

```
DATA_DIR = ”../../data”
OUTPUT_DIR = ”Output”

rule CountAlice:
    input: DATA_DIR + ”/canterbury/alice29”
    output: OUTPUT_DIR + ”/alice29.count”
    shell:
        ”””
        cat {input} | head -n 1000 | wc -w -c >{output}
        ”””
```

←

Note: Variables are enclosed in curly braces.

[2] V2

# Sample Rule[2]

```
DATA_DIR = ”../../data”                                    ←
OUTPUT_DIR = ”Output”                                      ←

rule CountAlice:
    input: DATA_DIR + ”/canterbury/alice29”
    output: OUTPUT_DIR + ”/alice29.count”
    shell:
        ”””
        cat {input} | head -n 1000 | wc -w -c >{output}
        ”””
```

Note: Variables are enclosed in curly braces.

[2] V2

MANCHESTER 1824
The University of Manchester

# Sample Rule[3]

```
$ ls
Snakefile
$ snakemake --cores 1 Output/alice29.count
Building DAG of jobs...
Using shell: /usr/bin/bash
Provided cores: 1 (use --cores to define parallelism)
Rules claiming more threads will be scaled down.
Job stats:
job           count
----------  -------
CountAlice        1
total             1

Select jobs to execute...

[Mon Nov  9 21:01:49 2026]
rule CountAlice:
    input: ../../data/canterbury/alice29
    output: Output/alice29.count
    jobid: 0
    reason: Missing output files: Output/alice29.count
    resources: tmpdir=/tmp

[Mon Nov  9 21:01:49 2026]
Finished job 0.
1 of 1 steps (100%) done
Complete log: .snakemake/log/2026-11-09T210148.734900.snakemake.log
$ cat Output/alice29.count
   8457   47564
$
```

[3] V2

MANCHESTER 1824
The University of Manchester

# Example with two rules[4]

```
DATA_DIR = "../../data"
OUTPUT_DIR = "Output"

rule TopAlice:
  input:  DATA_DIR + "/canterbury/alice29"
  output:  OUTPUT_DIR + "/01-top/alice29.head"
  shell:
    """
    head -n 1000 {input} >{output}
    """

rule CountAlice:
  input:  OUTPUT_DIR + "/01-top/alice29.head"
  output:  OUTPUT_DIR + "/02-count/alice29.count"
  shell:
    """
    wc -w -c {input} >{output}
    """
```

[4] V3

MANCHESTER 1824
The University of Manchester

# Sample Rule[5]

```
$ ls
Snakefile
$ snakemake --cores 1 Output/02-count/alice29.count
Building DAG of jobs...
Using shell: /usr/bin/bash
Provided cores: 1 (use --cores to define parallelism)
Rules claiming more threads will be scaled down.
Job stats:
job           count
----------  -------
CountAlice        1
TopAlice          1
total             2

Select jobs to execute...

[Mon Nov  9 20:59:16 2026]
rule TopAlice:
    input: ../../data/canterbury/alice29
    output: Output/01-top/alice29.head
    jobid: 1
    reason: Missing output files: Output/01-top/alice29.head
    resources: tmpdir=/tmp

[Mon Nov  9 20:59:16 2026]
Finished job 1.
1 of 2 steps (50%) done
Select jobs to execute...

[Mon Nov  9 20:59:16 2026]
rule CountAlice:
    input: Output/01-top/alice29.head
    output: Output/02-count/alice29.count
    jobid: 0
    reason: Missing output files: Output/02-count/alice29.count; Input files updated by another
job: Output/01-top/alice29.head
    resources: tmpdir=/tmp

[Mon Nov  9 20:59:16 2026]
Finished job 0.
2 of 2 steps (100%) done
Complete log: .snakemake/log/2026-11-09T205916.516490.snakemake.log
$ cat Output/02-count/alice29.count
 8457 47564 Output/01-top/alice29.head
$
```

MANCHESTER 1824
The University of Manchester

# Generalising[6]

```
##  Include important variables
include:  "global-vars.smk"

rule all:
  input:
    OUTPUT_DIR + "/02-count/alice29.count",
    OUTPUT_DIR + "/02-count/asyoulik.count",
    OUTPUT_DIR + "/02-count/lcet10.count",
    OUTPUT_DIR + "/02-count/plrabn12.count"

rule FileTop:
  input:  DATA_DIR + "/canterbury/{fn}"
  output:  OUTPUT_DIR + "/01-top/{fn}.head"
  shell:
    """
    head -n 1000 {input} >{output}
    """

rule Count:
  input:  OUTPUT_DIR + "/01-top/{fn}.head"
  output:  OUTPUT_DIR + "/02-count/{fn}.count"
  shell:
    """
    wc -w -c {input} >{output}
    """
```

[6] V4

MANCHESTER 1824
The University of Manchester

# Generalising[7]

In this example,

- The path variables have been moved to a separate file called `global-vars.smk`. That file has been included in.
- There is an all rule which has an input directive but no output directive.
  - If it is first in the file, then it will be applied if you run Snakemake with no options. (i.e., a “default” rule)
  - Other rules can occur before it if they are included in (i.e., not before it in the same file).
- Together, we can now process all four files by just adding them to the all rule.

[7] V4

MANCHESTER 1824
The University of Manchester

# Generalising Results[8]

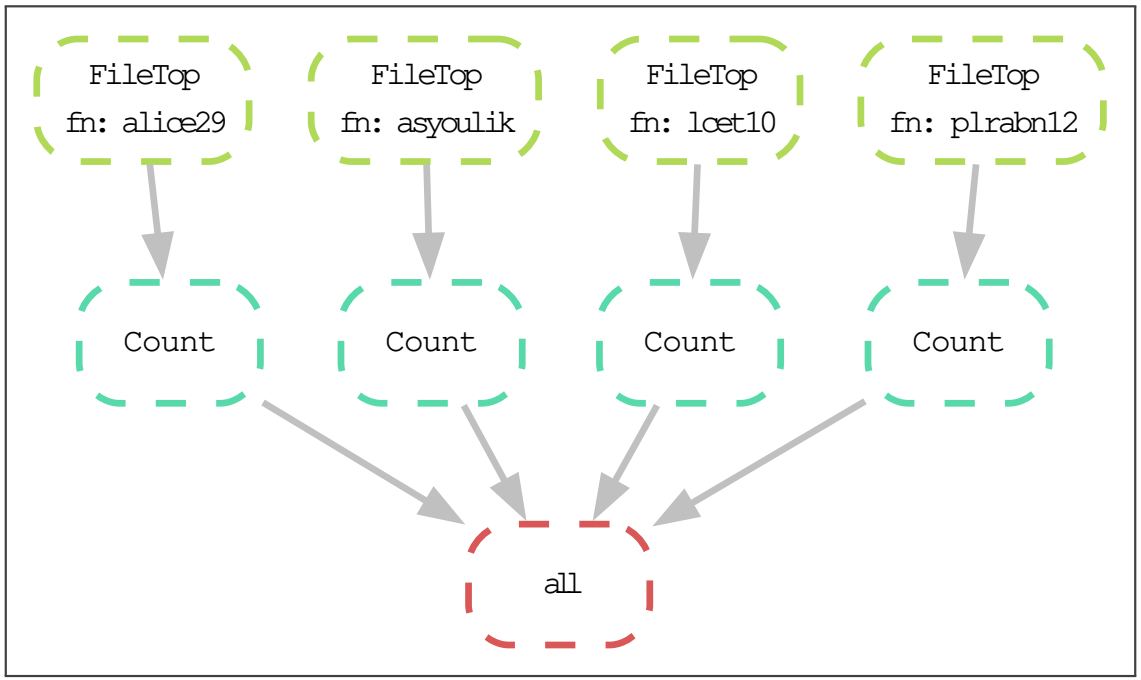
```
$ ls Output/02-count/
alice29.count  asyoulik.count  lcet10.count  plrabn12.count
$ cat Output/02-count/*
 8457 47564 Output/01-top/alice29.head
 5562 30667 Output/01-top/asyoulik.head
 7882 55658 Output/01-top/lcet10.head
 7401 44838 Output/01-top/plrabn12.head
$
```

[8] V4

# Visualisation[9]

Create a “DAG” (directed acyclic graph) for `graphviz` (not part of Snakemake):

```
snakemake --cores 1 --dag | dot -Teps >dag.eps
```




[9] V4

MANCHESTER 1824
The University of Manchester

# Python instead of bash commands[10]

```
DATA_DIR = "../../data"
OUTPUT_DIR = "Output"

rule CountAlice:
  input:  DATA_DIR + "/canterbury/alice29"
  output:  OUTPUT_DIR + "/alice29.count"
  run:
    with open (output[0], "w") as outfp:
      with open (input[0]) as infp:
        num_words = 0
        num_chars = 0
        for line in infp:
          wordslist = line.split ()
          num_words = num_words + len (wordslist)
          num_chars += (sum (len (word) for word in wordslist))
      outfp.write (str (num_words))
      outfp.write ("\t")
      outfp.write (str (num_chars) + "\t" + output[0] + "\n")
```

[10] V5

MANCHESTER 1824
The University of Manchester

# Outline

Introduction

Background

Basic structure

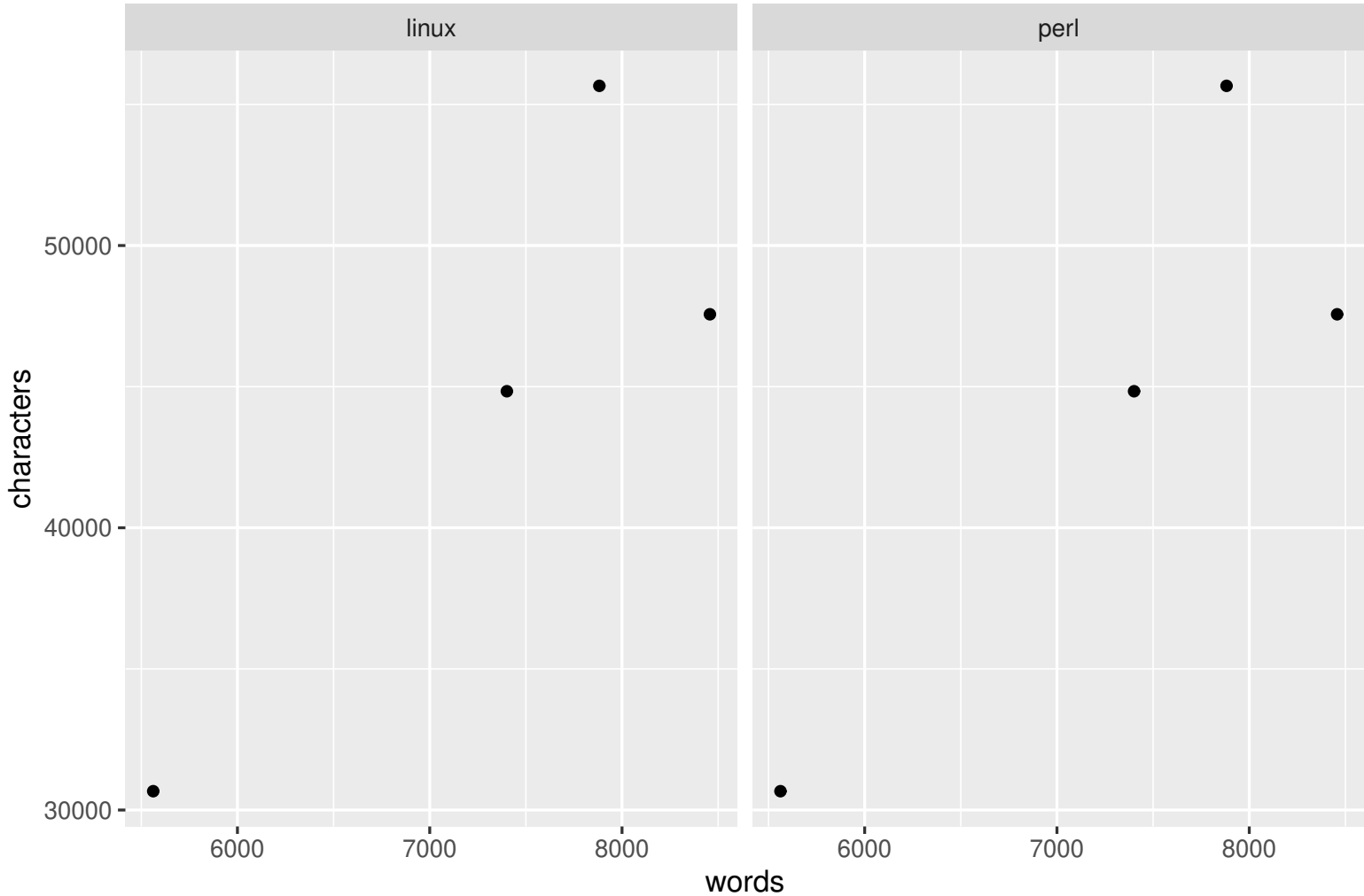
Final example

Tips

Summary

MANCHESTER 1824
The University of Manchester

# Graph



MANCHESTER 1824
The University of Manchester

# Data table

Create a data table in tab-separated format: `fileslist.tsv`.

| ID | corpus | filename |
|---|---|---|
| 1 | calgary | book1 |
| 2 | calgary | book2 |
| 3 | calgary | news |
| 4 | calgary | progc |
| 5 | calgary | progl |
| 6 | calgary | progp |
| 7 | canterbury | alice29 |
| 8 | canterbury | asyoulik |
| 9 | canterbury | lcet10 |
| 10 | canterbury | plrabn12 |

MANCHESTER 1824
The University of Manchester

# Main Snakefile[11]

```
import pandas as pd

from snakemake.utils import validate, min_version

configfile: "config.yml"
validate (config, schema = "config.schema.yml")

fileslist_panda = pd.read_table (config["fileslist"], sep='\t', lineterminator='\n')
fileslist_panda.set_index ("ID", drop=False, inplace=True)
validate (fileslist_panda, schema="fileslist.schema.yml")

##  Define global constraints on wildcards
wildcard_constraints:
  corpus = "calgary|canterbury"

##  Include important variables
include:  "global-vars.smk"

include:  "single.smk"
include:  "combine.smk"
include:  "graphs.smk"

rule all:
  input:
    OUTPUT_DIR + "/05-graphs/canterbury.jpg"
```

[11] V6

MANCHESTER 1824
The University of Manchester

# graphs.smk[12]

```
rule Scatterplot:
  input:
    input_fn1 = OUTPUT_DIR + "/04-combine-perl/{corpus}.tsv",
    input_fn2 = OUTPUT_DIR + "/04-combine-linux/{corpus}.tsv",
  output:
    output_fn1 = OUTPUT_DIR + "/05-graphs/{corpus}.jpg",
    output_fn2 = OUTPUT_DIR + "/05-graphs/{corpus}.png",
    output_fn3 = OUTPUT_DIR + "/05-graphs/{corpus}.eps"
  params:
    corpus = "{corpus}",
    myeps = lambda wildcards: config["my_options"]["postscript"]
  log:
    log_fn1 = OUTPUT_DIR + "/05-graphs/{corpus}.log"
  shell:
    """
    ##  {params.corpus} is available
    mypng="png"
    {TOOLS_DIR}/scatterplot.R --type jpg --perl {input.input_fn1} --linux
      {input.input_fn2} --output {output.output_fn1} >{log.log_fn1} 2>&1
    {TOOLS_DIR}/scatterplot.R --type ${{mypng}} --perl {input.input_fn1} --linux
      {input.input_fn2} --output {output.output_fn2} >>{log.log_fn1} 2>&1
    {TOOLS_DIR}/scatterplot.R --type {params.myeps} --perl {input.input_fn1} --linux
      {input.input_fn2} --output {output.output_fn3} >>{log.log_fn1} 2>&1
    """
```

[12] V6

MANCHESTER 1824
The University of Manchester

# combine.smk[13]

```
rule CombinePerl:
  input:  GatherPerl
  output:  OUTPUT_DIR + "/04-combine-perl/{corpus}.tsv"
  shell:
    """
    cat {input} >{output}
    """
```

[13] V6

MANCHESTER 1824
The University of Manchester

# combine.smk[14]

```
def GatherPerl (wc):
  results = []  ##  Initialise empty list

  ##  Iterate through each row of the table
  for row in fileslist_panda.itertuples (index = False):
    curr_ID = str (row.ID)  ##  Convert integer to string
    curr_corpus = row.corpus  ##  Current "corpus" in this row
    curr_filename = row.filename

    ##  Check for equality between wildcard corpus and current corpus
    if wc.corpus == curr_corpus:
      d = [OUTPUT_DIR + "/02-count-perl/{f}.count".format (f=curr_filename)]
      results.extend (d)  ##  Add it to the list

  ##  Print out the results
  print (results, file=sys.stderr)

  return results
```

[14] V6

# combine.smk[14]

```
def GatherPerl (wc):
  results = []  ##  Initialise empty list

  ##  Iterate through each row of the table
  for row in fileslist_panda.itertuples (index = False):
    curr_ID = str (row.ID)  ##  Convert integer to string
    curr_corpus = row.corpus  ##  Current "corpus" in this row
    curr_filename = row.filename

    ##  Check for equality between wildcard corpus and current corpus
    if wc.corpus == curr_corpus:
      d = [OUTPUT_DIR + "/02-count-perl/{f}.count".format (f=curr_filename)]
      results.extend (d)  ##  Add it to the list

  ##  Print out the results
  print (results, file=sys.stderr)

  return results
```

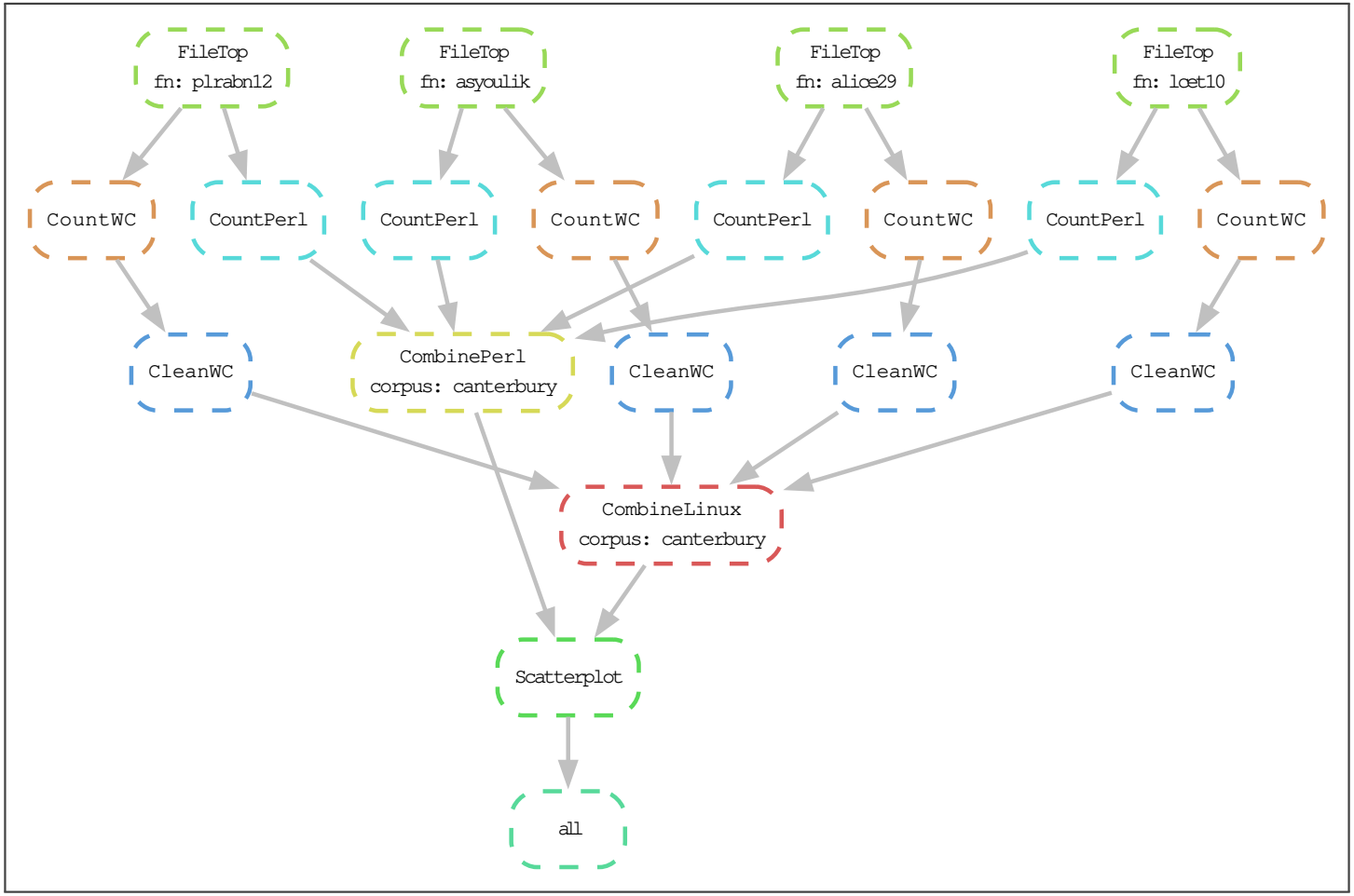
The `print ()` produces this:

```
['Output/02-count-perl/alice29.count', 'Output/02-count-perl/asyoulik.count',
'Output/02-count-perl/lcet10.count', 'Output/02-count-perl/plrabn12.count']
```

[14] V6

MANCHESTER 1824
The University of Manchester

# Visualisation[15]




[15] V6

MANCHESTER 1824
The University of Manchester

# Update the all rule

This can also be done to the all rule to process both corpora:

```
rule all:
  input:
    OUTPUT_DIR + "/05-graphs/canterbury.jpg",
    OUTPUT_DIR + "/05-graphs/calgary.jpg"
```

MANCHESTER 1824
The University of Manchester

# Some things to note

```
rule Scatterplot:
  input:
    input_fn1 = OUTPUT_DIR + "/04-combine-perl/{corpus}.tsv",
    input_fn2 = OUTPUT_DIR + "/04-combine-linux/{corpus}.tsv",
  output:
    output_fn1 = OUTPUT_DIR + "/05-graphs/{corpus}.jpg",
    output_fn2 = OUTPUT_DIR + "/05-graphs/{corpus}.png",
    output_fn3 = OUTPUT_DIR + "/05-graphs/{corpus}.eps"
  params:
    corpus = "{corpus}",
    myeps = lambda wildcards: config["my_options"]["postscript"]
  log:
    log_fn1 = OUTPUT_DIR + "/05-graphs/{corpus}.log"
  shell:
    """
    ##  {params.corpus} is available
    mypng="png"
    {TOOLS_DIR}/scatterplot.R --type jpg --perl {input.input_fn1} --linux
      {input.input_fn2} --output {output.output_fn1} >{log.log_fn1} 2>&1
    {TOOLS_DIR}/scatterplot.R --type ${{mypng}} --perl {input.input_fn1} --linux
      {input.input_fn2} --output {output.output_fn2} >>{log.log_fn1} 2>&1
    {TOOLS_DIR}/scatterplot.R --type {params.myeps} --perl {input.input_fn1} --linux
      {input.input_fn2} --output {output.output_fn3} >>{log.log_fn1} 2>&1
    """
```

MANCHESTER 1824
The University of Manchester

# Some things to note

- R script is meant to run on the command-line with supplied arguments (i.e., not interactively in RStudio).
- There is a log file! If a rule fails, Snakemake deletes all output generated (i.e., in the output directive). Those sent to the log directive are not. Looking at them will save you many headaches.
- I have given names to the input, output, and log directives. Without names and as a list, you can also refer to files with an index: `input[2]` (i.e., the third file in the input directive)

MANCHESTER 1824
The University of Manchester

# Some things to note

From `config.yml`:

```
fileslist:  "fileslist.tsv"

my_options:
  postscript:  "eps"
```

- Note 3 ways the argument to `--type` is provided and used:
  1. `jpg` as literal text
  2. `png` as a bash variable
  3. `eps` as a Snakemake variable which is stored in a file called `config.yml`. Note the use of `lambda` to access the dictionary created by the configuration file.

MANCHESTER 1824
The University of Manchester

# Much larger Snakemake example

## TargetMITO: a rule-based model for generating functional synthetic mitochondrial targeting sequences in yeast

**Kewin Gombeau [1,*], Raymond Wan [1,*], Joshua S. James [1], Déborah Tribouillard-Tanvier [2], Yizhi Cai [1,*]**

[1]Manchester Institute of Biotechnology, University of Manchester, 131 Princess Street, M1 7DN, Manchester, United Kingdom
[2]CNRS, IBGC, UMR 5095, University of Bordeaux, F-33000 Bordeaux, France

*To whom correspondence should be addressed. E-mail: yizhi.cai@manchester.ac.uk
Correspondence may also be addressed to Raymond Wan. E-mail: raymond.wan@manchester.ac.uk
Correspondence may also be addressed to Kewin Gombeau. E-mail: kewin.gombeau@manchester.ac.uk

MANCHESTER 1824
The University of Manchester

# Larger Snakemake example

GitHub repository (https://github.com/the-cai-lab/synMTS) has 4 workflows meant to run in order:

1. generate – Generate synthetic mitochondria targeting sequences (synMTS)
2. evaluate – Evaluate these sequences
3. statistics – Calculate some statistics over the properties of the synMTS
4. select – Take a subset of the results to focus on [for the manuscript]

MANCHESTER 1824
The University of Manchester

# Outline

Introduction

Background

Basic structure

Final example

Tips

Summary

MANCHESTER 1824
The University of Manchester

# Building a workflow

1. I "looked down" on `bash` scripts before, but you might want to start from that on one data file first as a starting point. Convert this to a Snakemake file.
2. Because of how wildcards are carried backwards (i.e., the corpus variable), you might want to start connecting things up from the all rule.
3. The Linux `touch` command can be useful here. It updates the date/time or creates a new file with 0 bytes with the current date and time.
   - So, you can link up the workflow first with `touch {output}` and then write the actual commands afterwards.
4. I started numbering my output directories to help me figure out the order of the output files.
5. Send error messages of programs to the log directive. If a rule fails, then you have a message to read.

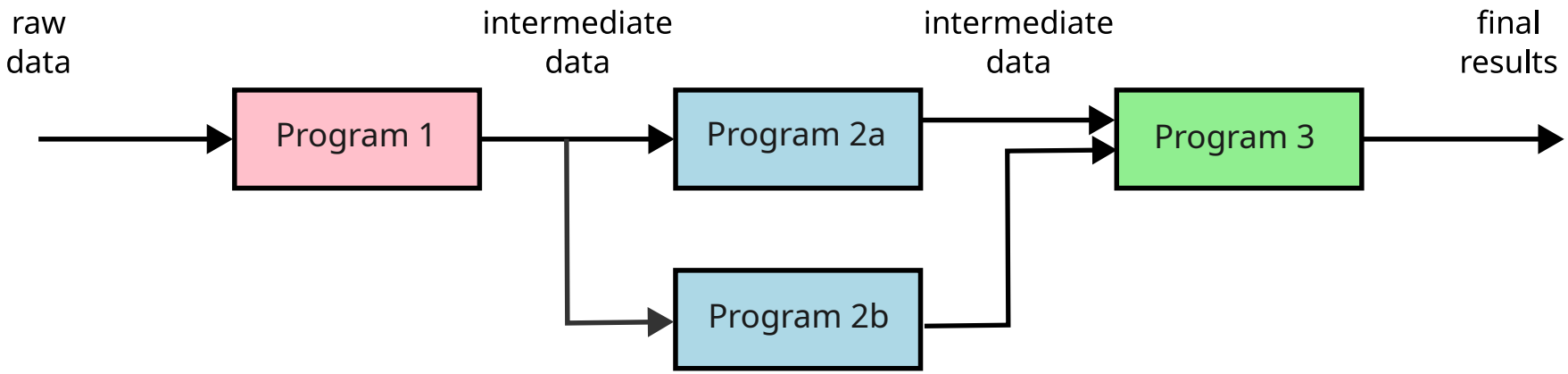
MANCHESTER 1824
The University of Manchester

# Testing a workflow

Use:

- a small test file.
- a single core (`--cores 1`) when testing. With more cores, the rules are executed simultaneously. Thus, the output will interleave and you can't tell which rule output what!
- the `-p` option so that it prints the command that it is running.

# Debugging a workflow




When trying to find out where in your workflow is causing you grief, you can remove an entire branch by commenting it out from the input directive of the rule it is going to.

Snakemake seems to:

1. Start from the end to the beginning, looking for which paths to follow.
2. When the execution paths are known, it runs it in the forward direction from the beginning.

MANCHESTER 1824
The University of Manchester

# Common error message

IncompleteFilesException:
The files below seem to be incomplete. If you are sure that certain files are not incomplete, mark them as complete with

snakemake –cleanup-metadata <filenames>

To re-generate the files rerun your command with the –rerun-incomplete flag. Incomplete files:
...

Snakemake stores its progress in a run in a “hidden” directory called `.snakemake`. If it starts getting its state wrong (i.e., you think it completed its last run, but it does not think so), then delete this directory and all of the output. Then, start again.

MANCHESTER 1824
The University of Manchester

# Outline

Introduction

Background

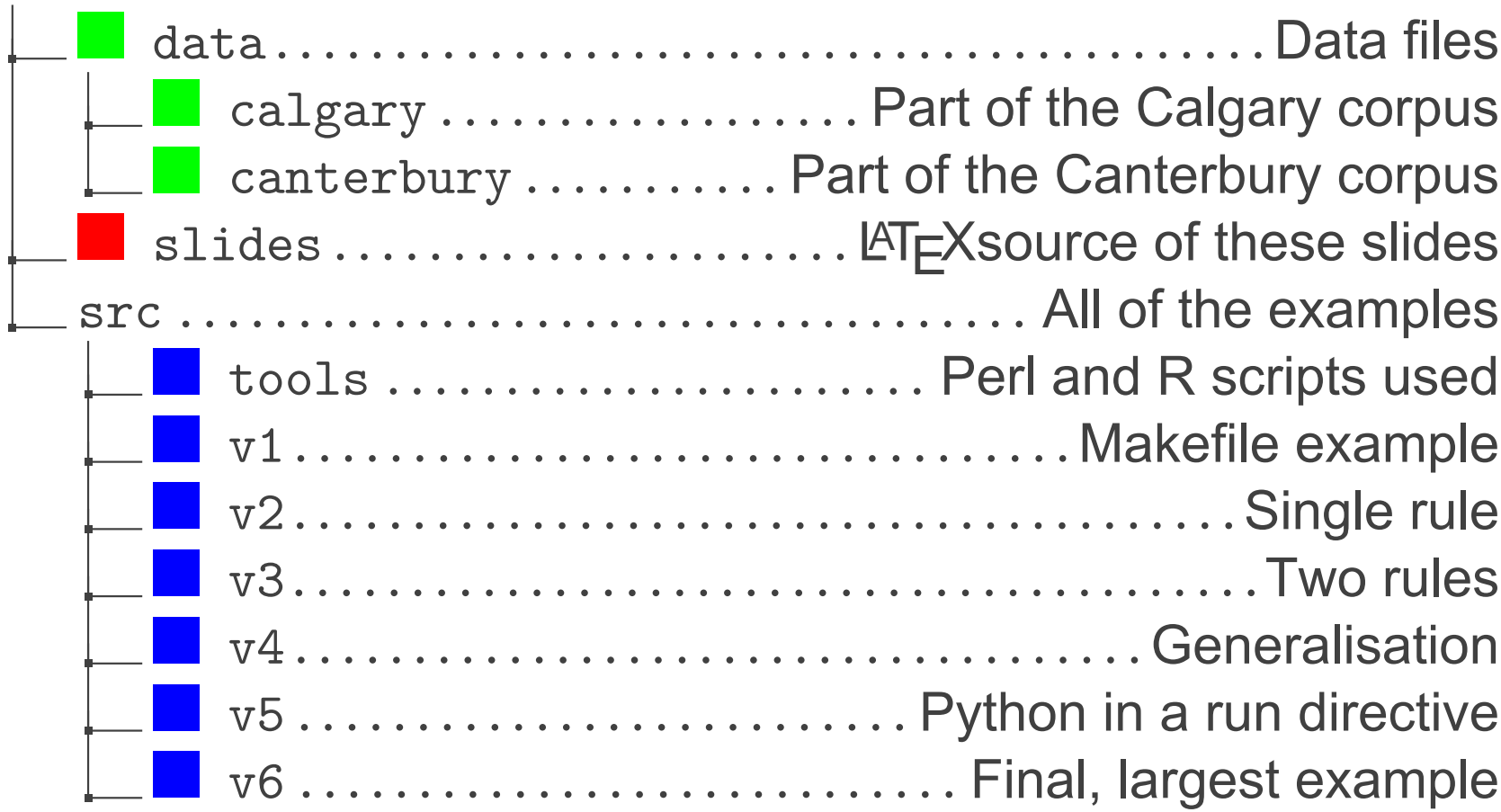
Basic structure

Final example

Tips

Summary

# GitHub repository




The PDF of the slides are at the top directory.

MANCHESTER 1824
The University of Manchester

# Snakemake = Python

→ Not really...

Since the developers made Snakemake with Python, there are many opportunities to leverage Python.

- A Python function to return a list of inputs paths.
- A `run` directive (or `script` directive) that accepts Python code.

But, you can write a Snakemake workflow without calling any Python code.

MANCHESTER 1824
The University of Manchester

# Snakemake = "reproducibility"

→ Unfortunately, no...

We are all looking for that "magical" method. If this method is used, then reproducibility of our analysis is "guaranteed".

Snakemake is a step in the right direction. But using it alone does not guarantee reproducibility.

MANCHESTER 1824
The University of Manchester

# Analogy

Snakemake defines a workflow, which is analogous to a recipe for baking. If you follow it, you will get the same cake every time.

But suppose the recipe asks for whole milk and, for whatever reason, whole milk is no longer available anywhere. You will need to substitute it with skim milk or oat milk (for example). Even if you still use the same quantity as the recipe, the end result will not be the same.

Something outside of the recipe (and beyond your control) has made reproducibility not possible.

MANCHESTER 1824
The University of Manchester

# More concrete example

This appears in one of the Perl scripts today:

```
use AppConfig;
use AppConfig::Getopt;
```

It allows a Perl script to process arguments on the command-line. On my system, version 1.71-2.3build1 of AppConfig was used. Years later, this version might be too old and version 2.00 (for example) might be needed.

There are ways to lock (or pin) libraries down to an exact version to address this problem. But if the system administrator tells you that version 1.71-2.3build1 has a serious security flaw and “you must upgrade” or “you will not be allowed to run it”, then you have no choice.

MANCHESTER 1824
The University of Manchester

# Support of Jupyter notebooks?

→ Unfortunately, no...

I am not an expert on Jupyter notebooks, but they seem to be very good at interactive analysis of data. Similar to RStudio.

With Snakemake, you need to write scripts (i.e., Python, R, etc.) that are not interactive. Parameters can be supplied to affect how the program is run.

# Support for SLURM/CSF3?

→ Yes!

Over 5 years ago, I created Snakemake scripts that would send $x$ (which you specify) jobs to SLURM. As your my jobs completed, new ones are added so that there are always around $x$ jobs running / queued.

Looking at the documentation, the method to do this appears to have changed. Now, it might called executors.

The problem for CSF3 is that you need your Snakemake script to be continuously running on the login node.

# Support for SLURM/CSF3

## E-mail from September 2026:

> Dear Dr Wan,
>
> Re: ServiceNow request item RITM[redacted]
>
> Yes, it is acceptable, at present, to use Snakemake on CSF3. We are unable to support it, however, so configuring it would need to be done by you. I have added the package to a list that I must look such that we might support it in the future.
>
> While not identical, we have notes on Nextflow that might cover some of the same topics (though that isn't a guarantee).
>
> Yours sincerely,

Link in message: https://ri.itservices.manchester.ac.uk/csf3/software/applications/nextflow/

MANCHESTER 1824
The University of Manchester

# Summary

- If you're using `bash` scripts or copy-and-pasting commands in from a lab notebook for a workflow consisting of many steps, you should consider some workflow-based system.
- Snakemake is one of many options.
- Of course, if you only run only a few commands, then Snakemake might be more than what you need!

MANCHESTER 1824
The University of Manchester

# Acknowledgements


- LaTeX template for these slides due to Alex Casson, which in turn is due to Andrew Mundy.
- QR code generation due to Adobe Express web site.
- Image generation / conversion: Gimp and Inkscape.
- Test data files: Calgary corpus (T. C. Bell, Cleary, and Witten, 1990) and Canterbury corpus (Arnold and T. Bell, 1997). Both were obtained from https://corpus.canterbury.ac.nz/.

MANCHESTER 1824
The University of Manchester

# Questions?

E-mail: raymond.wan@manchester.ac.uk

Computational Biology at Manchester Teams; Scripting and Coding channel:



MANCHESTER 1824
The University of Manchester

# Questions?



These slides



NAR paper

MANCHESTER 1824
The University of Manchester

# References I


- Arnold, R. and T. Bell (Mar. 1997). “A Corpus for the Evaluation of Lossless Compression Algorithms”. In: *Proc. 1997 IEEE Data Compression Conference*. Ed. by J. A. Storer and M. Cohn. Snowbird, Utah: IEEE Computer Society Press, Los Alamitos, California, pp. 201–210.
- Bell, T. C., J. G. Cleary, and I. H. Witten (1990). *Text Compression*. Prentice Hall.
- Köster, J. and S. Rahmann (2012). “Snakemake – a scalable bioinformatics workflow engine”. In: *Bioinformatics* 28.19, pp. 2520–2522.
- Mölder, F. et al. (2025). “Sustainable data analysis with Snakemake”. In: *F1000 Research* v3.